EXPOSICIÓN DE PAPER

A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting

Conteo de referencias concurrente: sin candados, sin esperas y sin memoria extra

Paper
Ivo Gabe de Wolff, Daniel Anderson, Gabriele K. Keller y Aleksei Seletskiy
Universidad de Utrecht · Carnegie Mellon University
Euro-Par 2024 · LNCS 14803 · págs. 103–118 · doi 10.1007/978-3-031-69583-4_8

Expositores
David · Hugo · Diego

Plan de la exposición

	Expone	Contenido	Tiempo
1	**David**	El problema: conteo de referencias, la carrera *read-reclaim*, garantías de progreso, lo que había y qué aporta el paper. *Demo: la carrera con AddressSanitizer.*	$\approx$ 10 min
2	**Hugo**	La solución: conteo ponderado, algoritmo 1 (`store`/`load`), empaquetado en 64 bits, la recarga *wait-free* (algoritmo 2) y un ejemplo con números. *Demo: el simulador encuentra los fallos.*	$\approx$ 10 min
3	**Diego**	Corrección y evaluación: demostración, coste, extensiones, experimentos del paper, *nuestras pruebas en local*, crítica y conclusiones.	$\approx$ 10 min

Pruebas en local: Intel i5-10400 (12 hilos), WSL Ubuntu 24.04, g++ 13.3, código original de los autores.

Una analogía para toda la exposición

Analogía	Término del paper	Qué es de verdad
Libro	objeto	datos en el montón
Hoja de usuarios	*reference count* (rc)	contador junto al objeto
Pase en tu mano	*shared pointer* (peso 1)	referencia propia e inmutable
Cartel en la pared	*atomic shared pointer*	variable compartida y mutable
Talonario del cartel	*weight* (peso)	bits altos del puntero
Bibliotecario / lector	`store` / `load`	escritor / lector
Tirar el libro	*reclaim*	liberar la memoria

La idea del paper: **el cartel lleva un talonario de tickets ya apuntados en la hoja**. Leer = arrancar un ticket.

Parte 1

El problema

Expone: **David**

Conteo de referencias · la carrera *read-reclaim* · lo que había

El paper en una frase

¿Cómo leer un puntero compartido a un objeto con conteo de referencias **mientras otro hilo lo cambia**, sin candados, sin que nadie espere y sin gastar memoria extra?

- Euro-Par 2024 (LNCS 14803), Madrid.
- Utrecht (de Wolff, Keller) y CMU (Anderson, Seletskiy).
- Sello *Artifact Evaluated*: código y *benchmark*s públicos (Zenodo y GitHub).
- Mejora la técnica *split reference count*, que se usa en producción (Folly, de Facebook) pero solo es *lock-free*.
- La hace ***wait-free*** y la explica de forma más sencilla.

Conteo de referencias en 30 segundos

```
auto a = std::make_shared<Libro>(); // rc = 1
{
    auto b = a;       // copia:       rc = 2
}                     // b se destruye: rc = 1
a = nullptr;          // rc = 0: se libera
```

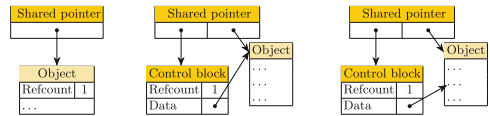
Se usa en `std::shared_ptr` (C++), `Arc` (Rust), Swift, CPython.

- Copiar una referencia: **rc += 1**. Destruirla: **rc −= 1**.
- rc llega a 0 ⇒ se libera **en ese momento**.
- Ventaja: si rc = 1, el dueño único puede **modificar en el sitio**.
- Problemas clásicos: ciclos (punteros débiles) y **concurrencia** — el tema del paper.

Shared pointer y atomic shared pointer

- **Shared pointer**: puntero *inmutable* con conteo. Su contador ya es seguro con hilos.
- **Atomic shared pointer**: celda *mutable* que contiene un shared pointer; varios hilos la leen (`load`) y la cambian (`store`). En C++20: `std::atomic<std::shared_ptr<T>>`.

```
std::atomic<std::shared_ptr<Config» config;
auto c = config.load();   // lector: su pase
config.store(std::make_shared<Config>(n));
```



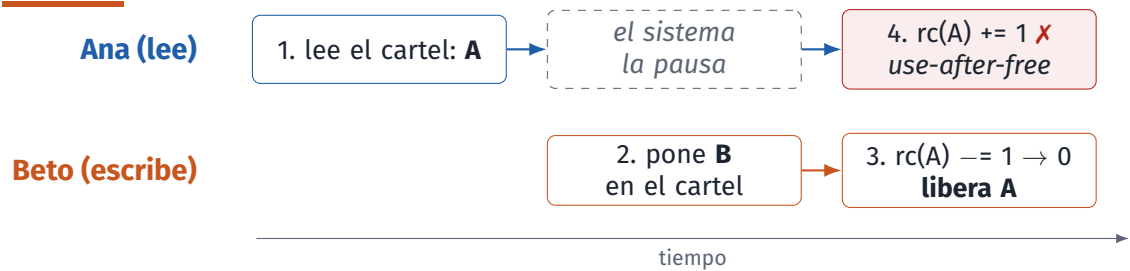
(a) Stored in the object

(b) Stored in control block, without aliasing

(c) Stored in control block, with aliasing

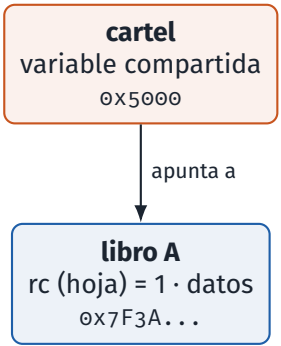
Fig. 1 del paper: dónde se guarda el contador. El algoritmo se explica con (a); (b) es igual con una indirección; (c) *aliasing*, en la sección 5.2.

La carrera *read-reclaim*



- Leer el puntero y apuntarse en el contador son **dos pasos**; entre ellos, otro hilo puede liberar el objeto.
- La ventana dura nanosegundos: el fallo es rarísimo, pasa las pruebas y aparece en producción. Es una causa típica de vulnerabilidades.

¿Por qué no hacerlo en un solo paso?



Puntero y contador viven en **dos sitios distintos** de la memoria.

Instrucción atómica	¿Existe hoy?
`fetch_and_add`, `swap`, CAS (64 bits)	✓
CAS doble anchura (2 palabras *contiguas*)	✓
DCAS (2 palabras *separadas*)	✗
`fetch_and_add` de 128 bits	✗

Una instrucción atómica abarca **una palabra**: no puede leer el cartel y modificar la hoja a la vez.

Garantías de progreso

Garantía	Definición (Herlihy, 1991)	Ejemplo: sumar 1 a un contador
Bloqueante	un hilo puede esperar indefinidamente a otro	protegerlo con un candado
Lock-free	en todo momento **algún** hilo termina en un número finito de pasos	bucle CAS: si mi CAS falla es porque otro tuvo éxito, pero yo podría fallar siempre
Wait-free	**cada** hilo termina en un número **acotado** de sus propios pasos	`fetch_and_add`: una instrucción, pase lo que pase

¿Por qué exigir *wait-free* aquí? La gestión de memoria está en todas partes: si una lectura de un puntero compartido puede atascarse, cualquier parte del programa puede atascarse. *Wait-free* da un tiempo máximo garantizado y latencias predecibles.

Lo que había

Solución	*Wait-free*	Sin mem. extra	Hw. actual	Precio
Candado (`std::atomic<shared_ptr>` de GCC)	✗	✓	✓	bloqueante
Valois; Michael y Scott (1995)	✗	✗	✓	gestor propio
Detlefs, Martin, Moir y Steele (2001)	✗	✓	✗	necesita DCAS
Herlihy et al. (2005); Gidenstam et al. (2009)	✗	✗	✓	reclamación diferida
Sundell (2005)	✓	✓	✓	cada CAS cuesta $\Theta(P)$
Anderson, Blelloch y Wei, CDRC (2021–22)	—	✗	✓	reclamación diferida
Split reference count (Williams, Folly)	✗	✓	✓	*lock-free*
Este paper	✓	✓	✓	CAS: *lock-free*

Exigencias: que funcione, que nadie espere, que no gaste memoria y que sea rápido. Ninguna solución anterior cumplía las cuatro.

Contribuciones

1. Un *atomic shared pointer* **wait-free** para `load`, `store` y `swap`, sin reclamación diferida y sin memoria extra.
2. `store` en tiempo constante; `load` casi siempre **una instrucción atómica** y, en el peor caso, $O(P)$ con P = hilos que usan esa variable.
3. *Compare-and-swap* y punteros *aliased*: también soportados, pero solo *lock-free*.
4. Reexplica el *split reference count* como un caso de **conteo de referencias ponderado**: una explicación (y una demostración) más sencilla.
5. Experimentos: el más rápido de los que no gastan memoria extra y la menor latencia de lectura.

Demo 1: la carrera existe (AddressSanitizer)

WSL · Ubuntu 24.04 · Intel Core i5-10400 (12 hilos)

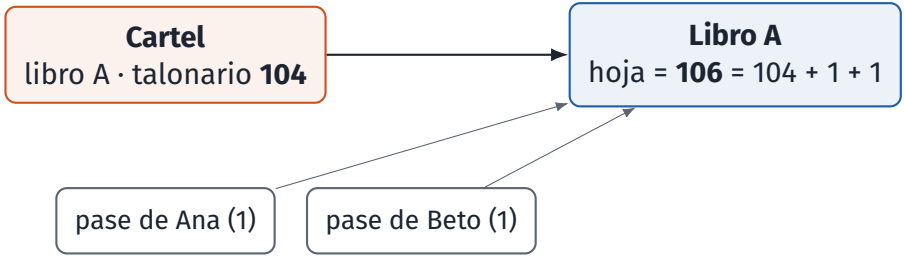
```
$ ./bin/estres_asan ingenua 2>&1   # (salida filtrada)
==493==ERROR: AddressSanitizer: heap-use-after-free en 0x502000001410
WRITE of size 8 at 0x502000001410 thread T1
    #0 std::__atomic_base<long>::fetch_add(long, std::memory_order) atomic_base.h:635
    #1 Ingenua::load() estres_asan.cpp:29
    #2 operator() estres_asan.cpp:44
freed by thread T2 here:
    #0 operator delete(void*, unsigned long) asan_new_delete.cpp:164
    #1 Ingenua::soltar(Libro*) estres_asan.cpp:24
    #2 Ingenua::store(Libro*) estres_asan.cpp:25
```

WSL · Ubuntu 24.04 · Intel Core i5-10400 (12 hilos)

```
$ ./bin/estres_asan autores
version=autores  hilos=8  operaciones=1600000 (10 % store, 90 % load)
terminado en 56 ms sin errores de memoria (suma de comprobacion 60464082)
```

8 hilos, 10 % store / 90 % load en la misma celda. Arriba: versión ingenua. Abajo: código de los autores.

La idea, antes de pasar a Hugo



- Los tickets del talonario **ya están apuntados** en la hoja del libro: son referencias pagadas por adelantado.
- Leer = **arrancar un ticket** con un `fetch_and_add`, que a la vez dice qué libro hay. La hoja no se toca.
- Leer y apuntarse pasan a ser **un solo paso**: la ventana peligrosa desaparece.

Parte 2

La solución

Expone: **Hugo**

Conteo ponderado · algoritmo 1 · empaquetado · recarga *wait-free*

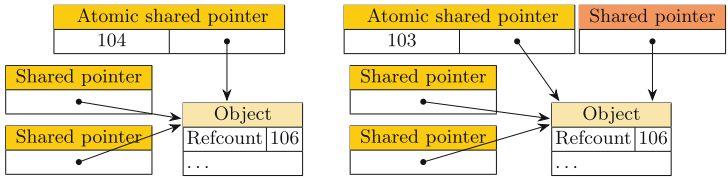
Conteo de referencias ponderado (Bevan; Watson y Watson, 1987)

- Cada referencia tiene un **peso**; el contador es la **suma de pesos**.
- Soltar: restar su peso.
- **Clonar**: repartir el peso, **sin tocar el contador**.
- Clonar una de peso 1 exige subir antes su peso.

Acción	Pesos	Hoja	¿Toca la hoja?
inicio	X = 10	10	—
clonar X en Y	X = 6, Y = 4	10	**no**
tirar Y	X = 6	6	sí (−4)
clonar X en Z	X = 3, Z = 3	6	**no**
tirar Z y X	—	0	libera

En memoria compartida, «tocar la hoja» es tocar una posición lejana: evitarlo es lo que **elimina la carrera**.

El peso en el *atomic shared pointer*



(a) Situation before a load.

(b) Situation after a load. The orange shared pointer (top right) is newly created.

Fig. 2 del paper. (a) 104 + 1 + 1 = 106. (b) Tras un `load`: el cartel baja a 103, aparece un pase nuevo y la hoja sigue en 106.

- Al crear el cartel se le da un peso $C \gg P$ y se suma C a la hoja. Cada pase pesa 1.
- Invariante: **hoja** $\geq$ **peso del cartel + pases**; igual cuando no hay operaciones a medias. Por arriba es seguro; por abajo, liberaríamos con lectores dentro.

Algoritmo 1: `store` y `load`

Algorithm 1. Reading from and writing to an atomic shared pointer

```
function STORE(atomic_shared_ptr<T>* this, shared_ptr<T> new)
    Increment reference count of new by C
    (previous_ptr, previous_weight) ← UNPACK(
        ATOMIC_SWAP(&this→value, PACK(new.pointer, C)))
    Decrement reference count of previous_ptr by previous_weight
function LOAD(atomic_shared_ptr<T>* this)
    (ptr, weight) ← UNPACK(
        ATOMIC_FETCH_AND_ADD(&this→value, −PACK(null, 1)))
    if weight − 1 < P then INCREASE_WEIGHT(this, ptr, weight − 1)
    return shared_ptr(ptr)
```

Algoritmo 1 del paper.

`store`

1. sumar *C* al objeto nuevo **antes** de publicarlo;
2. `swap`: poner (nuevo, *C*) y llevarse (viejo, *w*) de golpe;
3. restar al viejo **solo** el sobrante *w*.

`load`

1. `fetch_and_add(−pack(null,1))`: un ticket menos y el valor anterior, en **una instrucción**;
2. si quedan menos de *P*: recargar (algoritmo 2);
3. devolver un pase de peso 1, ya pagado.

Por qué desaparece la carrera

Caso 1: F antes que S2

Ana arranca un ticket: cartel (A, 4), Ana tiene un pase de A.
S2 recoge (A, 4); S3 resta 4: hoja de A = 5 − 4 = **1** = el pase de Ana.
A vive mientras Ana lo usa y se libera cuando lo suelta.

Caso 2: F después de S2

El cartel ya muestra (B, *C*): Ana se lleva un pase de **B**, respaldado desde S1.
S3 resta 5 a A: se libera sin nadie dentro.
Ana nunca toca A.

No hay tercer caso: el estado «he leído pero no me he apuntado» ya no existe.

Empaquetar puntero y peso en 64 bits

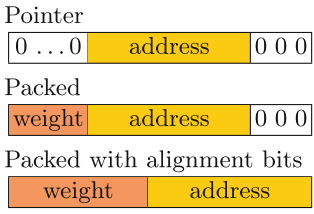


Fig. 3 del paper.

- x86-64 usa 48 bits de dirección: **16 bits altos libres** para el peso ($C \leq 2^{16} - 1$); con los 3 bits de alineación, 19 ($C \leq 2^{19} - 1$).
- Restar un ticket = restar 2^{48} a la palabra entera:

```
  0x0068 7F3A12345670
− 0x0001 000000000000
= 0x0067 7F3A12345670
```

- La dirección no cambia; el acarreo sale por arriba. Con `null`, el peso da la vuelta sin estropear el puntero.
- El contador del objeto, de 64 bits: con 32 bits y $C = 2^{19}$, bastarían 8192 carteles para desbordarlo.

Recargar el talonario: el punto delicado

Cuando un `load` deja el peso por debajo de P, **ese hilo** recarga: reserva $C - w$ en la hoja y sube el peso lo mismo. Dos trampas:

- el cartel pudo cambiar de libro (`store` intermedio);
- otro hilo pudo recargar ya (desbordamiento).

$\Rightarrow$ no vale `fetch_and_add`: hace falta un **CAS** («solo si sigue como lo vi»).

Requisitos para ser *wait-free*

R-1: $C \geq 2P$.

R-2: el peso solo **baja de 1 en 1** o se **fija por encima de** P.

Con R-1, tras recargar:
actual $+ C - w \;\geq\; 0 + 2P - (P - 1) = P + 1 > P$.

Consecuencia de R-2: si el CAS falla y el peso **subió**, alguien ya recargó $\Rightarrow$ **salida temprana**.

La idea informal: Figura 4

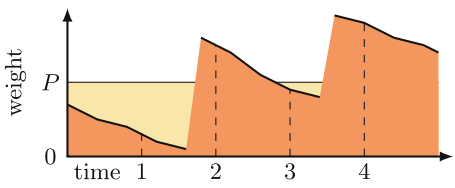


Fig. 4 del paper: peso de un cartel en el tiempo; la línea es *P*.

1. Un hilo empieza un `load` con el peso bajo: va a recargar.
2. Otro hilo recarga antes: el peso salta por encima de *P*.
3. El CAS del primero falla. El peso vuelve a ser bajo, pero **mayor que el que vio**: por R-2, alguien lo subió por encima de *P*. Puede irse.
4. Quienes lo bajaron después de 2 completan la siguiente recarga.

Quien mete el talonario en la zona roja, lo saca; si otro se adelanta, se va.

Algoritmo 2: `increase_weight`

Algorithm 2. Wait-free method to increase the weight of atomic shared pointers.

```
function INCREASE_WEIGHT(atomic<size_t>* value, ptr, weight)
    Increment reference count of ptr by C − weight
    current_weight ← weight
    loop
        cas ← ATOMIC_COMPARE_AND_SWAP(value,
              PACK(ptr, current_weight), PACK(ptr, current_weight + C − weight))
        if cas.success then return
        (new_ptr, new_weight) ← UNPACK(cas.original)
        if new_ptr ≠ ptr or new_weight > current_weight then break
        current_weight ← new_weight
    Decrement reference count of ptr by C − weight
```

Algoritmo 2 del paper.

1. reservar $C - w$ en la hoja **antes** del CAS;
2. CAS: si el cartel sigue en (ptr, actual), sumar $C - w$;
3. éxito: terminar;
4. otro puntero o **peso mayor**: devolver la reserva y salir;
5. peso menor: reintentar con el nuevo valor.

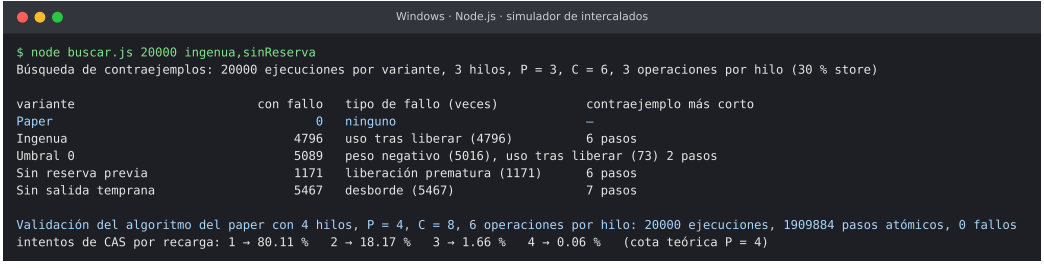
Los pesos esperados forman una cadena **estrictamente decreciente** entre $P - 1$ y 0: **como mucho P intentos**.

Ejemplo completo: $P = 4$, $C = 8$

Paso	Qué pasa	Tickets	Hoja	Pases	O	$P - \lvert O \rvert$
1	`store`: cartel con 8, hoja con 8	8	8	0	$\emptyset$	4
2	cuatro lecturas normales ($8 \to 4$)	4	8	4	$\emptyset$	4
3	Ana lee $4 \to 3$: **ofensa**; recargará con $w = 3$	3	8	5	{Ana}	3
4	Beto lee $3 \to 2$: ofensa; recargará con $w = 2$	2	8	6	{Ana, Beto}	2
5–6	Ana reserva 5; Beto reserva 6	2	19	6	{Ana, Beto}	2
7	Ana: CAS($3 \to 8$) falla; ve $2 < 3 \to$ reintenta	2	19	6	{Ana, Beto}	2
8	Ana: CAS($2 \to 7$) éxito; O se vacía	7	19	6	$\emptyset$	4
9	Beto: CAS($2 \to 8$) falla; ve $7 > 2 \to$ **sale**	7	19	6	$\emptyset$	4
10	Beto devuelve su reserva de 6	7	**13**	6	$\emptyset$	4

Al final: 7 tickets + 6 pases = 13 = hoja. La hoja nunca va por debajo.

Demo 2 (1/2): un simulador encuentra los fallos de cada variante



Simulador de intercalados (paso atómico a paso atómico) que comprueba los invariantes tras cada paso.

Paper: 0 fallos; cada variante rota falla en 2 a 7 pasos.

Demo 2 (2/2): traza de un contraejemplo

Windows · Node.js · simulador de intercalados

```
$ node buscar.js 20000 ingenua,sinReserva   # (continuación)
Contraejemplo más corto — Recargar el cartel antes de apuntar la reserva
paso  hilo    acción                                                cartel      hojas
   0  —       estado inicial                                        (L1, 3)     L1=3
   1  Beto    crea L2 (hoja 1: su propio pase)                      (L1, 3)     L1=3 L2=1
   2  Beto    apunta +6 en la hoja de L2                            (L1, 3)     L1=3 L2=7
   3  Carla   load: arranca 1 ticket (3→2), ofensa; le toca recargar (L1, 2)     L1=3 L2=7
   4  Carla   CAS 2→6: éxito                                        (L1, 6)     L1=3 L2=7
   5  Beto    swap: cartel L1→L2, se lleva 6 sobrantes              (L2, 6)     L1=3 L2=7
   6  Beto    resta 6 a la hoja de L1 → L1 liberado                 (L2, 6)     L1=† L2=7
→ LIBERACIÓN PREMATURA: L1 se liberó con 1 referencia todavía en uso.
```

Intercalado mínimo que rompe una variante, con el estado tras cada paso atómico.

Parte 3

Corrección y evaluación

Expone: **Diego**

Demostración · coste · extensiones · experimentos · pruebas en local

La demostración: el peso nunca es negativo

Por qué importa: el peso está en 16 bits sin signo; -1 sería 65 535 *tickets fantasma* sin respaldo $\Rightarrow$ *use-after-free*.

Ofensa: decrementar el peso cuando vale $\leq P$.
O: multiconjunto de ofensores; se vacía cuando alguien fija el peso por encima de P.

Invariante 1: cada hilo está en O como mucho una vez (quien ofende recarga, y no sale sin que O se vacíe).

Invariante 2: peso $\geq P - |O|$ (desde el último vaciado solo baja de 1 en 1; cada bajada en zona roja suma 1 a $|O|$).

$$\text{peso} \;\geq\; P - |O| \;\geq\; P - P \;=\; 0$$

Por eso el umbral de recarga es P y no 0: es el colchón para que todos ofendan una vez a la vez.

Coste

- El bucle CAS solo repite si **otro** hilo bajó el peso: la cadena $w_0 > w_1 > \cdots \geq 0$, con $w_0 \leq P - 1$, tiene como mucho P elementos.
- Peor caso de `load`: $O(P)$, con P = hilos que **usan esa variable**.
- Amortizado: tras recargar quedan $\approx C$ tickets; con $C = 2^{19}$ y $P = 2^{16}$ hay $\approx$ 458 752 lecturas entre recargas (2 por millón).

Operación	Coste	Garantía
`store`, `swap`	$O(1)$	*wait-free*
`load`	1 instr.; $O(P)$ peor	*wait-free*
compare-and-swap	sin cota	*lock-free*
`load` *aliased*	sin cota	*lock-free*

Extensiones (sección 5)

- **`null`**: con el peso en los bits altos, restar tickets a `null` solo da la vuelta al peso; no hace falta recargar.
- **Punteros débiles**: ortogonales; misma técnica.
- **Compare-and-set**: bucle CAS sobre la palabra (libro, tickets). Cualquier lectura cambia los tickets y hace fallar al escritor ⇒ solo *lock-free*.
- ***Aliasing***: dos punteros en 128 bits; camino rápido con `fetch_and_add` en los primeros 64 y, si hay *aliasing*, bucle CAS de 128 bits (*lock-free*).

Algorithm 3. Compare-and-set on atomic shared pointers

```
function COMPARE-AND-SET(atomic_shared_ptr<T>* this,
        shared_ptr<T> expected, shared_ptr<T> new)
    Increment reference count of new by C
    (observed_ptr, observed_weight) ← UNPACK(*this→value)
    loop
        if observed_ptr ≠ expected.pointer then
            Decrement reference count of new by C; return false
        cas ← ATOMIC_COMPARE_AND_SWAP(&this→value,
            PACK(observed_ptr, observed_weight), PACK(new.pointer, C))
        if cas.success then
            Decrement ref count of observed_ptr by observed_weight; return true
        (observed_ptr, observed_weight) ← UNPACK(cas.original)
```

Algoritmo 3 del paper.

Experimentos del paper: montaje

- $C = 2^{19}$, $P = 2^{16}$ (R-1 con holgura).
- Rendimiento: 4× Xeon E7-8867, 72 núcleos, 144 hilos.
- Latencia: Xeon w7-3465X, 28 núcleos, 56 hilos.
- **Pila concurrente**: n pilas, u % de actualizaciones (sacar de una pila y meter en otra con *compare-and-set*); el resto lee la cima (`load`).

Rival	Estrategia
Libstdc++ (GCC)	candados
Williams	*split refcount*
Folly (Facebook)	*split refcount*
Anderson	*hazard pointers*

Figura 5: rendimiento

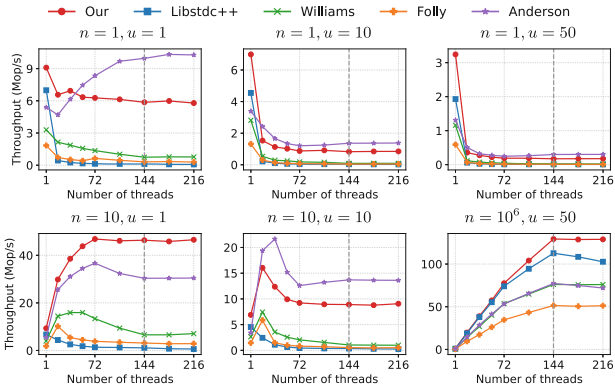


Fig. 5 del paper: millones de operaciones por segundo frente a hilos; la línea discontinua marca 144.

- Con 1 hilo, **Our** es la más rápida en todos los paneles.
- Gana siempre a los rivales sin memoria extra (GCC, Williams, Folly): con 1 pila y 144 hilos, ≈7× Williams y ≈20× Folly.
- Frente a Anderson depende de la **contención**: gana con $n = 10, u = 1$ y con 10^6 pilas; pierde con una sola pila (las actualizaciones usan el CAS *lock-free*).

Figura 6: latencia

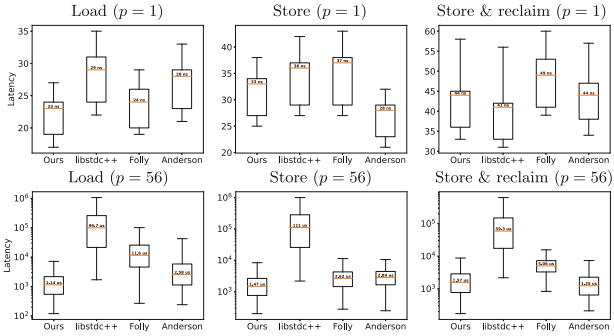
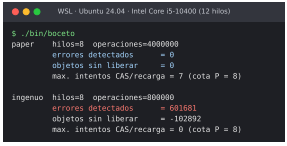


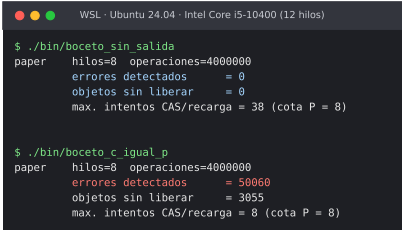
Fig. 6 del paper: percentiles 25, 50 y 75 (caja) y 1 y 99 (bigotes); con $p = 56$, escala logarítmica.

`load`, $p = 56$	mediana
Ours	**1,14 µs**
Anderson	2,58 µs
Folly	11,6 µs
Libstdc++	99,7 µs

- `load`: el menor siempre, y con la cola más corta.
- `store`: 2.º con $p = 1$, 1.º con $p = 56$.

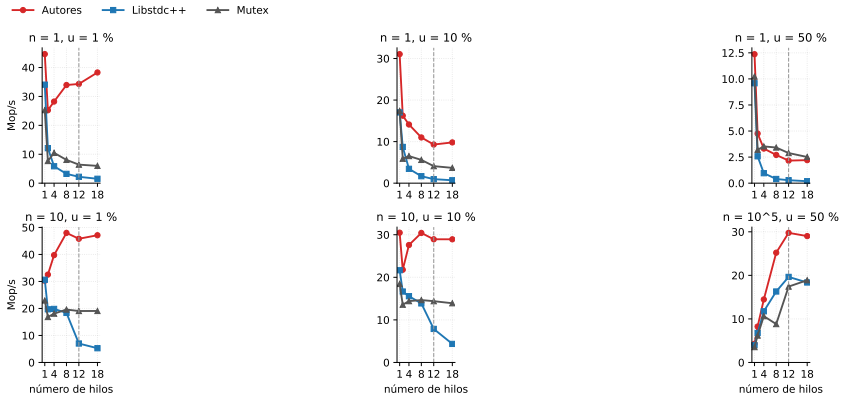
Pruebas en local (1): el boceto y sus variantes rotas





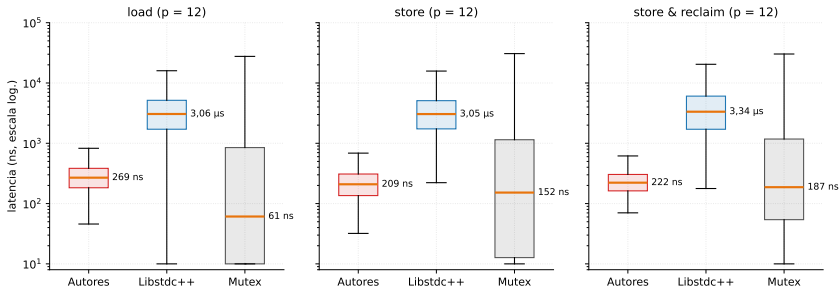
- Boceto en C++, $P = 8$, $C = 16$. Paper: **0 errores**, como mucho 7 intentos ($\leq P$). Ingenua: 601 681 errores y dobles liberaciones.
- Sin salida temprana: 0 errores pero **38 intentos** $> P$ (no *wait-free*). Con $C = P$ (rompe R-1): 50 060 errores.

Pruebas en local (2): rendimiento con el código de los autores



Mini-Fig. 5: pila con el código original (Autores), `atomic<shared_ptr>` de GCC 13 (Libstdc++) y `std::mutex`. 1 s por punto; línea = 12 hilos; último panel $n = 10^5$.

Pruebas en local (3): latencia con 12 hilos



Mini-Fig. 6: 1 hilo mide y 11 hacen `store`; 100 000 muestras; percentiles 1/25/50/75/99.

`load`: Autores 269 ns (p99 0,83 µs) · Libstdc++ 3,06 µs (p99 16 µs) · Mutex 61 ns, pero **p99 27,6 µs**: la mediana engaña; la cola delata a quien espera.

¿Coincide con el paper?

Se reproduce

- El código de los autores: **0 errores** con AddressSanitizer; la versión ingenua cae enseguida.
- Con 1 hilo es la más rápida en los seis paneles.
- Gana con claridad a la versión con candados (GCC): con 1 pila, $u = 1$ y 12 hilos, 34,3 frente a 2,2 Mop/s.
- Latencia con contención: mediana y cola muy por debajo de GCC; cola mucho más corta que el mutex.

Matices

- Con 1 pila y 50 % de actualizaciones, el mutex la supera (2,9 frente a 2,2 Mop/s): el CAS *lock-free* reintenta, como reconoce el paper.
- 12 hilos, no 144: la contención es menor y no hay sobresuscripción grande.
- Sin Folly, Williams ni Anderson (dependencias pesadas); WSL en Windows.
- Mediciones de un solo equipo y una sola ejecución: tendencias, no cifras absolutas.

Relación con el *split reference count*

- *Split refcount*: un **contador local** junto al puntero, que **sube** desde 0 con cada `load` y se transfiere al objeto al cambiar el puntero.
- El paper cuenta lo contrario: el **peso baja** desde C.

$$\text{peso} = C - \text{contador local}$$

Lo nuevo no es la técnica, sino la recarga que **sale si otro ya recargó**: eso la hace *wait-free*.

Versión	`load`	Garantía
Lee (2010)	FAA doble palabra	no existe en hw
Williams	CAS doble anchura	*lock-free*
Folly	bucle CAS, $C = 8192$	*lock-free**
Paper	FAA de 1 palabra	***wait-free***

* Folly pasa a bloqueante con más de C hilos; usa un contador de 32 bits.

Valoración crítica

Fortalezas

- Sencillo: unas 30 líneas de núcleo y una demostración de una página.
- `load`/`store`/`swap` *wait-free*, sin memoria extra, liberación inmediata (rc = 1 fiable).
- Lecturas de una instrucción: menor latencia y cola corta.
- Mejora software real (Folly, GCC) y publica su código.

Límites

- CAS y *aliasing* solo *lock-free*.
- Cada lectura **escribe** en la variable compartida (no hay *snapshot pointers*).
- Límite estricto de P hilos: es de **corrección**, no de rendimiento.
- Depende de bits libres en el puntero (paginación de 5 niveles, ARM con PAC/MTE).
- No discute el orden de memoria; evalúa una estructura y una arquitectura.

Conclusiones

1. El conteo de referencias con variables mutables puede ser **wait-free sin retrasar la liberación**.
2. La clave: un **peso prepagado** en el propio puntero; leer y apuntarse son una sola instrucción.
3. La recarga termina en como mucho P intentos gracias a R-1 ($C \geq 2P$) y R-2 (salida temprana).
4. Funciona en la práctica: lo comprobamos en local con el código original (sin errores y mucho más rápido que la versión con candados bajo contención).

Para discutir

- ¿Se podría hacer el compare-and-swap *wait-free* (técnicas de ayuda entre hilos) sin perder rapidez?
- ¿Compensaría un modo de lectura sin escritura para variables que casi nunca cambian?

¿Preguntas?

David · Hugo · Diego

Código, capturas y gráficas de esta exposición: carpeta `exposicion_paper/` (`codigo/ejecutar_pruebas.sh`)

Referencias

- I. G. de Wolff, D. Anderson, G. K. Keller y A. Seletskiy. *A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting*. Euro-Par 2024, LNCS 14803, págs. 103–118. Springer. doi:10.1007/978-3-031-69583-4_8.
- Código de los autores: `github.com/ivogabe/wait-free-atomic-shared-pointer` (commit 8540c58); artefacto en Zenodo, doi:10.5281/zenodo.11775182.
- M. Herlihy. Wait-free synchronization. *ACM TOPLAS* 13(1), 1991.
- D. I. Bevan; P. Watson e I. Watson. Conteo de referencias ponderado. PARLE 1987.
- D. Detlefs, P. Martin, M. Moir y G. Steele. Lock-free reference counting. PODC 2001.
- H. Sundell. Wait-free reference counting and memory management. IPDPS 2005.
- D. Anderson, G. Blelloch e Y. Wei. Concurrent deferred reference counting with constant-time overhead. PLDI 2021.
- A. Williams. *C++ Concurrency in Action*. Manning, 2012.

Figuras y algoritmos «del paper»: reproducidos de la publicación original con fines académicos.

Anexo: cómo reproducir las pruebas

```
# en Linux o WSL (g++ >= 12), desde exposicion_paper/codigo
bash ejecutar_pruebas.sh        # compila y guarda todo en ../capturas
#   01 demo de los autores        02-03 AddressSanitizer (ingenua / autores)
#   04-05 boceto y variantes      06-07 rendimiento y latencia (CSV)

# simulador de intercalados (Node.js), desde exposicion_paper/simulador
node buscar.js 20000 ingenua,sinReserva

# graficas y capturas como PDF, desde exposicion_paper
python graficos/hacer_graficos.py
```

